

AI Model Zoo Module for SPIDHIVE

Nadine Arabel A. Anareta and Val Randolph M. Madrid

Abstract—The effective monitoring of crop pests and diseases is critical for agricultural sustainability in the Philippines. While applications like SPIDTECH+ facilitate remote diagnostics, the underlying infrastructure previously lacked a centralized mechanism for managing the iterative development of the machine learning models themselves. This study developed an AI Model Zoo integrated within the SPIDHIVE platform to streamline the storage, management, and deployment of agricultural AI models. A robust, bidirectional cross-platform architecture was engineered utilizing Supabase as a dedicated mobile backend. This integration enables the efficient downstream, over-the-air delivery of compiled `.tflite` models to the SPIDTECH+ client, while simultaneously facilitating the upstream mirroring of images for expert review. The web platform further optimizes developer workflows through automated artifact parsing, robust dataset tracking, and automated configuration generation to prevent class confusion during model retraining. System evaluation by AI Developers yielded an "Excellent" System Usability Scale (SUS) score of 88.0, confirming high functional efficiency and minimal cognitive load. Ultimately, this localized repository establishes a continuous improvement pipeline for agricultural AI, providing researchers with the necessary infrastructure to accelerate the deployment of accurate diagnostic tools for crop protection.

Index Terms—SPIDTECH+, SPIDHIVE, AI Model Repository, Cross-Platform Synchronization, Pest and Disease Identification

I. INTRODUCTION

A. Background of the Study

The Philippines is primarily considered an agricultural country, accounting for 7.5% of the total GDP during the second quarter of 2025 and contributing approximately 56% of the total value of agriculture and fisheries in the Philippines [1]. The agriculture sector provides livelihood and employment to most of the population, dominating the economic growth of the Philippines [2].

Problems and challenges in the agriculture sector are inevitable. Poor crop production has been a long-standing problem, primarily due to typhoons, limited adoption of high-yielding varieties, crop diseases, and pest infestations [3]. Pests and diseases affect the most widely produced crop commodities, such as rice, corn, and cassava [4]. Early detection of pests and crop diseases by identification and remote monitoring could be one way to alleviate problems in crop protection. Farmers, or non-experts, can help identify pests and diseases by gathering data to be assessed by experts or the citizen science approach [5].

A local initiative called Smarter Approaches to Reinvigorate Agriculture in the Philippines (Project SARAI) provides real-time monitoring for crops, including pests and diseases, which includes the Smarter Pest and Disease Identification

Technology (SPIDTECH) app [6]. SPIDTECH is a mobile application that provides real-time monitoring of pests and diseases in crops [6]. A web application that provides a real-time mapping and reporting tool, and acts as a repository for AI models that help identify crops and pests. However, this companion application has no crowd sourcing, which led to the development of SPIDTECH+. SPIDTECH+ is an enhanced version of SPIDTECH that employs a citizen science approach, involving non-experts in solving crop disease and pest infestation problems [7].

B. Statement of the Problem

The current web application, SPIDHIVE, serves as a repository for image datasets of crops, pests, and diseases, but lacks key features to support the storage, management, and sharing of AI models. To streamline the management of existing and the development of new AI models, there is a need to develop an AI model zoo within SPIDHIVE for use in SPIDTECH+. This platform will enable users to upload and download AI models, thereby accelerating access to resources and facilitating collaboration among researchers and practitioners, specifically from the National Crop Protection Center (NCPC).

C. Objectives of the Study

The study aims to enhance SPIDHIVE by developing a localized AI model zoo that enables efficient storage, management, and deployment of AI models for real-time pest and disease identification. Specifically, it seeks to:

- 1) Develop an AI model zoo module that enables storage, management, and sharing of AI models for pest and disease identification within SPIDHIVE.
- 2) Implement APIs to streamline access to AI models, ensuring seamless integration with SPIDHIVE
- 3) Integrate the AI model module zoo into the SPIDHIVE web application, ensuring seamless access and sharing of AI models.
- 4) Evaluate the web application in terms of functionality, usability, and performance.

D. Significance of the Study

The study provides a centralized and accessible AI model zoo within SPIDHIVE, allowing faster development and sharing of models for pest and disease identification. This supports innovation, reduces duplication of work, and empowers agricultural stakeholders with timely diagnostic tools. Furthermore, the study will serve as a reference for future developers and researchers in the enhancement, expansion, and advancement of agricultural monitoring systems.

E. Scope and Limitation

The study is limited to the design and development of the AI model zoo module of SPIDHIVE. The datasets used in training and testing the models is limited to the information provided by the National Crop Protection Center (NCPC).

II. RELATED REVIEW OF LITERATURE

A. Crop Diseases and Pest Monitoring

According to the Climate Change Commission [8], the agricultural sector has long been a crucial component of the Philippines' economy, contributing significantly to the country's GDP. However, in 2024 alone, the Philippines experienced agricultural losses due to natural disasters, including floods, droughts, pests, and diseases [9]. Staple crops such as rice and corn remain highly vulnerable, making timely detection and monitoring crucial to avoiding severe disruptions in food supply and financial stability.

Effective monitoring helps prevent widespread crop damage, yet conventional approaches remain limited. Observation through the naked eye, manual monitoring, or traditional human-based methods for symptoms of diseases are inefficient due to workload, fatigue, and human error [10].

Miranda et al. [11] employed image processing techniques to detect pest infestations in rice paddy fields. While Bengochea-Guevara et al. [12] implemented a complete drive-by-wire system using a commercial electric vehicle to create a fully autonomous agricultural inspection platform. In the Philippines, Medalla [13] introduced the application of the Wavelet Technique in Image Fusion and Reconstruction as an early detection method in reducing pests in the Philippines' agricultural sector.

B. AI Model for Crop Diseases and Pest Monitoring

Artificial intelligence has emerged as a promising solution for improving crop disease and pest identification and monitoring, ensuring a faster and more reliable system. AI models detect patterns from large datasets and apply these learned representations to make accurate predictions [14].

Zhang et al. [15] used the advancement of CNN in identifying cassava diseases by leveraging salient semantic features and promising high returns. Meanwhile, models such as YOLO, Faster R-CNN, and SSD were used by Raouhi et al. [16] in identifying pests that achieved a significant accuracy. Lu et al. [17] further enhanced the CNN for crop disease and pest identification, specifically on maize disease and Pentatomidae stinkbug pest. Similarly, Bijlwan et al. [18] used deep learning models to evaluate their efficiency and accuracy on detecting, classifying, and assessing diseases and pests on rice crops.

In the Philippines, Mique Jr. and Palaoag [19] successfully developed a CNN-based rice pest and disease detector using a dataset from the Regional Crop Protection Center, achieving 90.9% accuracy. A mobile application called SPIDTECH was also deployed for digital identification and remote monitoring of crop diseases and pests using a MobileNetV2 architecture [6].

C. Web and Mobile Applications for Crop Diseases and Pest Monitoring

With the existing monitoring applications, an accessible, fast, and advanced method using an affordable and AI-powered web and mobile application [6]. The Philippine Rice Research Institute (PhilRice) has also introduced eDamuhan as a mobile application that allows quick identification of weeds or pest and their consequent management [20]. On the other hand, SaBaTech is a web application for real-time detection of diseases and pests on bananas through image recognition using AI technology, alerting the farmers in real-time using SMS [21].

SPIDTECH, developed under the Smarter Approaches to Reininvigorate Agriculture as an Industry (SARAI) project, was trained on locally collected datasets of pests and diseases of rice, corn, coffee, cacao, banana, sugarcane, tomato, and soybean. Building upon SPIDTECH, SPIDTECH+ [7] expanded its coverage from six to twenty crops, such as mango, abaca, rubber, papaya, and ampalaya. This study also adopted a citizen science approach where non-experts, such as farmers, were allowed to upload images, which were later annotated by experts.

Both SPIDTECH and SPIDTECH+ are integrated with SPIDHIVE (CROPDEX), the centralized web platform that stores, visualizes, and manages pest and disease data. SPIDHIVE serves as the command and data center for the mobile applications: images uploaded from SPIDTECH are reflected in SPIDHIVE, where experts review and annotate them before they are used to retrain and improve AI models. The updated models are then deployed back into SPIDTECH+, ensuring a continuous improvement cycle between field data collection and AI model refinement [22].

D. Model Repositories and AI Model Zoo

AI model repositories are a well-curated collection of pre-trained models that developers can integrate into their applications without building from scratch [23]. Global platforms such as GitHub, Hugging Face Hub, TensorFlow Hub, and PyTorch Hub allow researchers to host pre-trained models with documentation, version histories, and standardized formats that support model reuse and reproducibility. However, because they are designed for broad and varied domains, they do not always meet the specialized requirements of agricultural applications, particularly those involving localized datasets, crop conditions, and field-use constraints.

Although many AI model repositories exist, they remain limited in addressing the specific needs of SPIDHIVE and its user base. SPIDHIVE already hosts agricultural datasets and supports users involved in pest and disease monitoring, yet it lacks a mechanism for directly integrating AI model management into its existing workflow. An internal AI model zoo would enable users to upload, download, update, and manage models that are directly aligned with the data already curated within the system. This localized repository prevents duplication of work, supports continuous improvement of existing models, and improves usability by keeping the entire pipeline within a single platform. Ultimately, establishing an

internal AI model zoo strengthens SPIDHIVE's role as a unified agricultural intelligence system, enabling more accurate diagnostics and contributing to more effective crop protection strategies in the Philippines.

III. METHODOLOGY

A. Sampling and Data Collection

The image datasets used for training and evaluating AI models were exclusively provided by the National Crop Protection Center (NCPC). As of May 2026, the centralized repository contains a total of 5,690 valid images and 23,639 expert-verified bounding box annotations spanning multiple crops. Shown in Figure 1 is a sample dataset of a corn that was uploaded, annotated, and reviewed by NCPC.

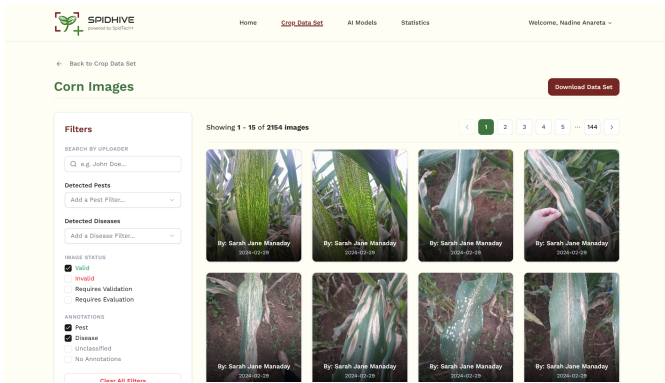


Fig. 1. Sample Corn Dataset

B. System Architecture and Design

The AI Model Zoo module leverages the existing SPIDHIVE framework to ensure seamless integration with current operations. The architecture follows a three-tier design pattern:

1) *Frontend Layer*: The client-side interface utilizes ReactJS with Vite to build a highly responsive, web-based management interface. The module incorporates modern user interface libraries, including Tailwind CSS, Lucide React, and Framer Motion for an intuitive user experience. The interface is designed to maintain consistency with existing SPIDHIVE components, utilizing a shared design system for data visualization and model management.

2) *Backend Layer*: The server-side application is developed using NodeJS and ExpressJS framework, providing a robust RESTful API. All API endpoints are designed to serve as a centralized hub, ensuring cross-platform data synchronization and seamless integration between the SPIDHIVE web application and SPIDTECH+ mobile application.

3) *Storage Layer and Database Model*: While the core SPIDHIVE production infrastructure is hosted on an Amazon Web Services (AWS) Lightsail instance, the AI Model Zoo module was developed and evaluated within a localized staging environment. This approach was strategically chosen to isolate complex architectural modifications and prevent potential disruptions to live system operations. The development server

was hosted locally, utilizing MySQL as the primary database management system to replicate the production schema. It employs the Sequelize Object-Relational Mapper (ORM) to enforce strict data integrity and manage complex relationships between developers, datasets, and AI models within this staging environment.

Furthermore, to securely and efficiently bridge this localized web repository with the SPIDTECH+ mobile application, Supabase is integrated as a dedicated mobile backend. Supabase's cloud storage buckets serve a dual purpose: they are utilized to host the compiled `.tf.lite` model artifacts for mobile deployment, and they store the raw image datasets that populate the SPIDTECH+ gallery, including the images uploaded from the field. To ensure the mobile app has immediate access to the latest repository updates during testing, essential AI model metadata is continuously synchronized from the local MySQL database to the Supabase database. This separation of concerns was established to create an architecture intended to prevent heavy mobile download traffic and high-volume image retrieval from bottlenecking the primary AWS web server upon final deployment.

C. Database Schema Implementation

The schema is designed to provide a comprehensive, traceable structure for managing the lifecycle of AI models and their corresponding training datasets. The following core entities were implemented to support the operations of the AI Model Zoo module:

1) *Dataset Registry (`cropdex_datasets`)*: This table functions as a historical ledger for every custom dataset generated by the system. It records the target crop, specific pest and disease labels requested, total image count, detailed metadata for the included images. This entity is linked to the requesting developer via the `creator_id` foreign key. Table 1 provides a comprehensive description of these dataset attributes.

TABLE I
DATASET METADATA

Field	Description
id	unique identifier of the dataset
crop	target crop of the dataset
labels	pests and diseases on the dataset
num_images	number of images on the dataset
image_ids	metadata of the images, which saves the id, split, label, annotations
creator_id	foreign key of the downloader
createdAt & updatedAt	dates created and updated

2) *AI Model Metadata (`cropdex_ai_models`)*: This primary entity stores the technical attributes and evaluation metrics of each AI model uploaded to the repository. It includes core attributes such as the model name, neural network architecture, and deployment status. To enforce integrity and track developer, it establishes a foreign key with the `cropdex_users` via `developer_id`. It also links to `cropdex_datasets` via `dataset_id` to explicitly map each model to the exact training data. For future expansion

and iterative model development, it also includes `parent_id` creating a traceable lineage of model versions. Table 2 details the complete attributes of this schema.

TABLE II
AI MODEL METADATA

Field	Description
<code>id</code>	unique identifier of the AI Model
<code>model_name</code>	name of the AI Model
<code>model_description</code>	description of the AI Model
<code>model_type</code>	classification whether it performs object detection or image classification
<code>crop</code>	target crop of the AI Model
<code>classes</code>	target pests and diseases of the model
<code>architecture</code>	neural network architecture
<code>version number</code>	current version that increments when re-trained
<code>status</code>	status whether an active or deleted model
<code>num_downloads</code>	number of downloads of the AI Model
<code>tags</code>	search tags
<code>developer_id</code>	foreign key of the developer
<code>dataset_id</code>	foreign key of the dataset used
<code>weights_file</code> , <code>tf_lite_file</code> , <code>config_file</code> , <code>metrics_file</code> , <code>evaluation_graphs</code>	files uploaded when zip file; accepts null when single file
<code>metrics_map50</code> , <code>metrics_precision</code> , <code>metrics_recall</code> , <code>metrics_accuracy</code> , <code>metrics_f1</code>	resulting metrics of the AI Model; map50 is only for detection, meanwhile accuracy is only for classification
<code>num_images</code>	number of images on the dataset used
<code>parent_model_id</code>	foreign key of the AI Model; null when original training
<code>createdAt & updatedAt</code>	dates created and updated

3) *SPIDTECH+ Registry (available_models)*: To decouple the mobile application's query load from the primary web database, a dedicated `available_models` table is maintained within the Supabase architecture. This table stores essential, mobile-optimized metadata, including the model identifier, name, target crop, version number, deployment status, and the last update timestamp. It records the download URL that points to the compiled `.tflite` artifact hosted within the Supabase `ai-models` storage bucket. This structure enables seamless, over-the-air model updates for the SPIDTECH+ application. Table 3 shows a detailed description of this entity.

TABLE III
AI MODELS METADATA IN SUPABASE

Field	Description
<code>model_id</code>	unique identifier of the model
<code>name</code>	name of the model
<code>crop</code>	target crop of the dataset
<code>version</code>	version number of the model
<code>download_url</code>	URL that points to the exact file in bucket storage
<code>updated_at</code>	date updated
<code>is_active</code>	status of the model

D. User Interface Design

The user interface for the AI Model Zoo is designed to maintain visual and functional consistency with the existing SPIDHIVE platform. To introduce specialized views for AI

model management, the design architecture draws inspiration from industry-standard machine learning repositories, such as Roboflow. This ensures a minimal learning curve for existing users while providing professional-grade model management features.

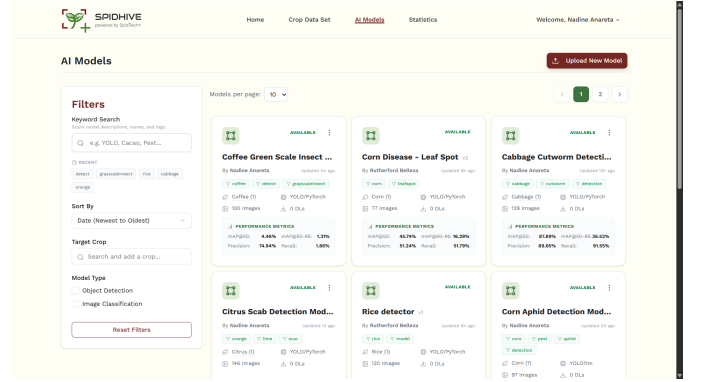


Fig. 2. AI Model Page

1) *AI Models Page*: The AI Models Page is a newly implemented module architected specifically to handle high-density data visualization and repository management.

Figure 2 illustrates the primary dashboard, comprising an upload interface, a robust multi-attribute filter sidebar, and a grid of interactive AI model cards. The system enables efficient retrieval through saved recent searches and comprehensive querying by developer, neural network architecture, deployment status, and target crops. Each model card previews essential metadata and provides deep-link navigation directly to specific module tabs, such as Images or Analytics. To ensure a modern aesthetic and functional consistency with the broader SPIDHIVE platform, all interface components utilize the Shaden UI Library and Tailwind CSS.

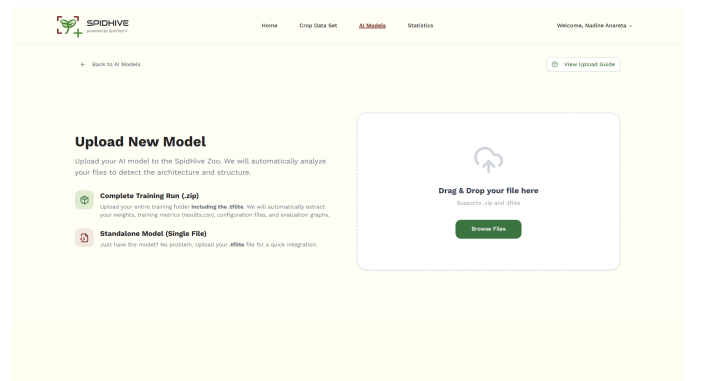


Fig. 3. Upload New AI Model

Figure 3 illustrates the upload interface, which features a drag-and-drop zone that accepts both compressed ZIP archives and standalone `.tflite` files for SPIDTECH+ compatibility. The system automatically detects essential files within a ZIP archive, such as the required `.tflite` model, weights, configurations, evaluation graphs, and performance

metric files. Furthermore, to facilitate seamless user onboarding, the interface integrates comprehensive instructional materials; developers can access video tutorials, web-based documentation, and downloadable PDF manuals via a dedicated help button strategically located at the top right of the module.

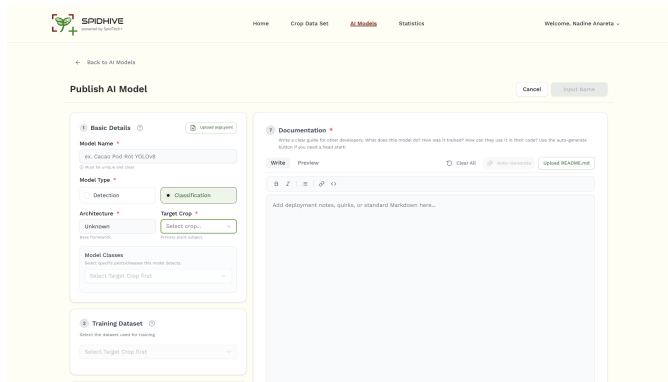


Fig. 4. Publish New AI Model

Figure 4 details the model publishing interface, designed to capture critical metadata and establish model lineage. To maximize system fault tolerance and accommodate varied developer workflows, the publishing interface was redesigned to feature segmented upload modules dedicated to basic details, performance metrics, and evaluation graphs. This modular architecture ensures that if an uploaded ZIP archive is incomplete or possesses an invalid directory structure, the automated parsing mechanism will not trigger a total system rejection; instead, it successfully extracts the available assets, such as the core `.tflite` artifact, while allowing developers to manually upload missing standalone files.

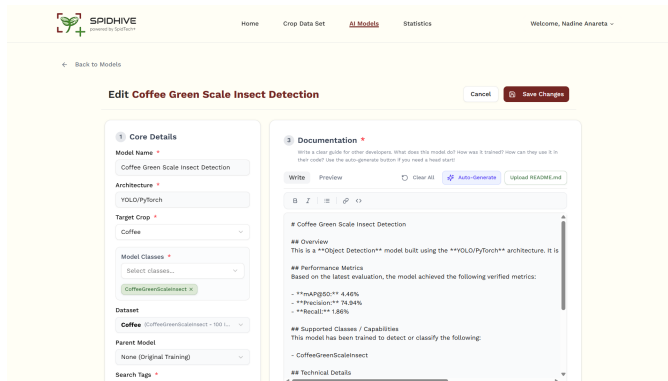


Fig. 5. Edit AI Model

Figure 5 illustrates the edit AI model interface, which auto-populates input fields with current metadata to preserve data continuity. Developers can modify key mutable attributes, including model name, architecture, target crop, dataset linkage, parent reference, and search tags, while utilizing advanced configuration tools like a visibility toggle to manage the public display of evaluation graphs. Furthermore, the interface retains the integrated rich-text editor, enabling developers to manually

format technical updates or utilize the "auto-generate" function to instantly refresh foundational documentation following architectural changes.

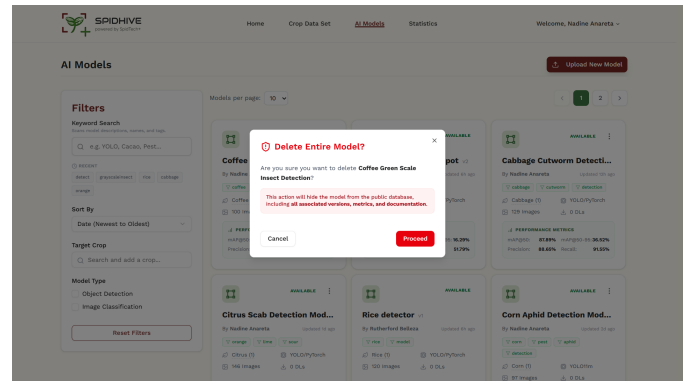


Fig. 6. Delete AI Model

Figure 6 shows the two-step confirmation process required to delete an AI model. To prevent accidental data loss, developers must enter their password to confirm the deletion. Additionally, to keep the system secure, only the original creator of a model has permission to edit or delete it.

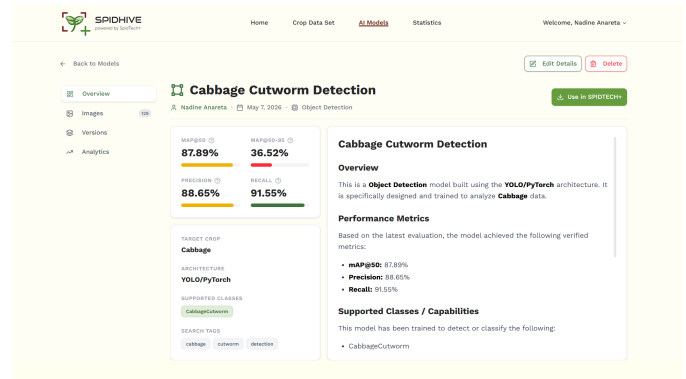


Fig. 7. AI Model Overview Tab

The AI model specification interface is organized into four functional tabs: Overview, Images, Versions, and Analytics. Figure 7 illustrates the Overview Tab, functioning as the primary dashboard to display critical metadata, including model name, contributing developer, performance metrics, target crop, neural network architecture, search tags, and technical documentation. To facilitate immediate deployment and streamline management, the page header centralizes retrieval and modification tools in a single view, granting users direct download access to the compiled `.tflite` artifact alongside the authorized developer's edit and delete functionalities.

Figure 8 illustrates the dataset gallery used to train the selected AI model. To help developers easily inspect the quality of the data, the interface allows users to dynamically filter the images by their designated machine learning split (training, validation, or testing) and by specific pest or disease categories. Furthermore, an interactive toggle lets users overlay

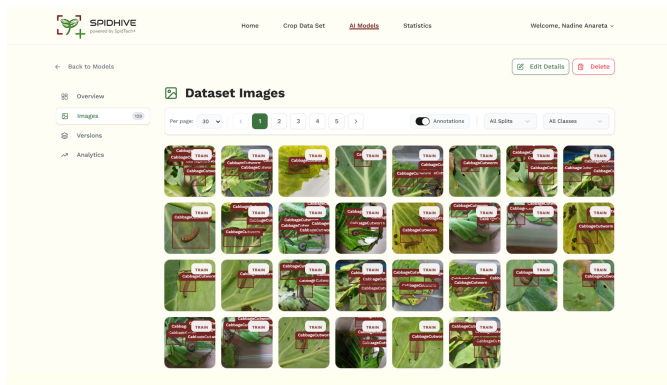


Fig. 8. AI Model Images Tab

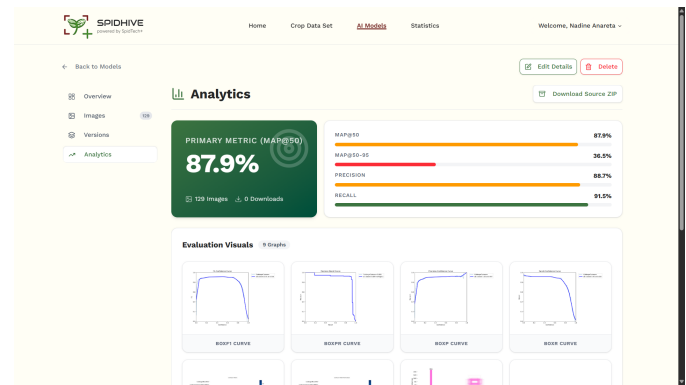


Fig. 10. AI Model Analytics Tab

the precise bounding box annotations directly onto the images. This feature provides a clear, visual confirmation of exactly what the AI was trained to detect or classify.

reproducibility.

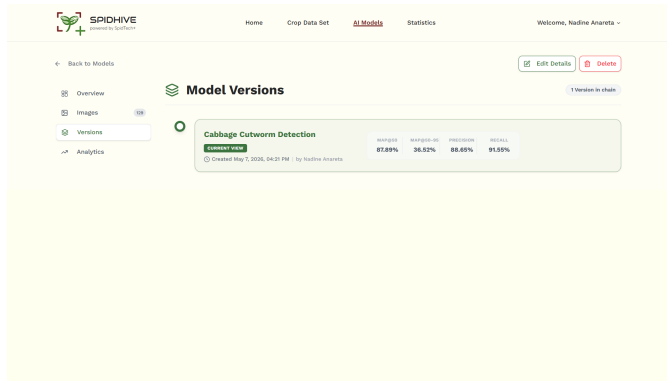


Fig. 9. AI Model Version Tab

Figure 9 illustrates the Versions Tab, which visualizes the developmental lineage of connected AI models. This interface clearly displays both the preceding "ancestor" models it was based upon and any subsequent "descendant" models that were fine-tuned from it. For every related model in this branch, the section presents essential metadata, including the model name, contributing developer, creation date, verified performance metrics, and current deployment status. To streamline the developer's workflow, each listed version includes quick-access action buttons. These allow users to instantly navigate to a related model's full specification page or directly download its `.tflite` file for mobile deployment.

Figure 10 illustrates the Analytics Tab, which provides a detailed visual dashboard of the AI model's performance metrics and evaluation graphs. To ensure thorough technical transparency, these analytical insights are automatically extracted and displayed exclusively for models uploaded via a complete ZIP archive. From this interface, users can also download the original source ZIP file. This feature provides researchers with direct access to all essential training artifacts, including the neural network weights, architectural configuration files, and raw data logs—enabling offline analysis and strict scientific

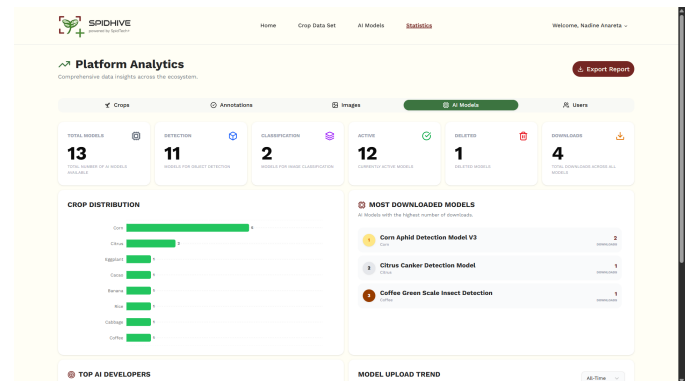


Fig. 11. Statistics Page

2) *Statistics*: Figure 11 illustrates the Statistics dashboard, a centralized interface for real-time system monitoring across five specialized tabs: Crops, Annotations, Images, AI Models, and Users. By integrating the `Recharts` library, the module intuitively visualizes complex data using Key Performance Indicator (KPI) cards, dynamic charts, and ranking tables to highlight top platform contributors. Additionally, a built-in export function enables administrators to download comprehensive, multi-sheet Excel reports detailing overall repository health and usage trends.

3) *Download Dataset*: Figure 12 illustrates the dataset download functionality, which compiles a structured ZIP archive by automatically separating raw images from their corresponding annotations. To mitigate model overfitting, the backend algorithmically partitions the data into randomized training, validation, and testing subsets based on user-defined distribution ratios. Crucially, the system auto-generates a `.yaml` configuration file using fixed numerical class indices; this strict mapping ensures the neural network consistently aligns the correct value with its specific agricultural label, actively preventing class confusion during iterative retraining.

4) *My Account*: Figure 13 illustrates the redesigned user profile, updated to ensure strict visual consistency with the AI Model Zoo. To enforce system security and workflow bound-

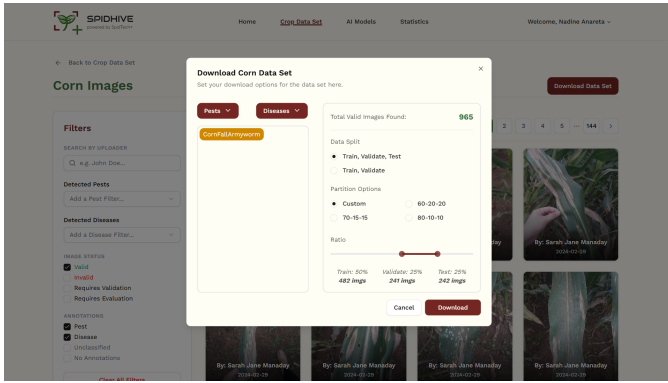


Fig. 12. Download Dataset

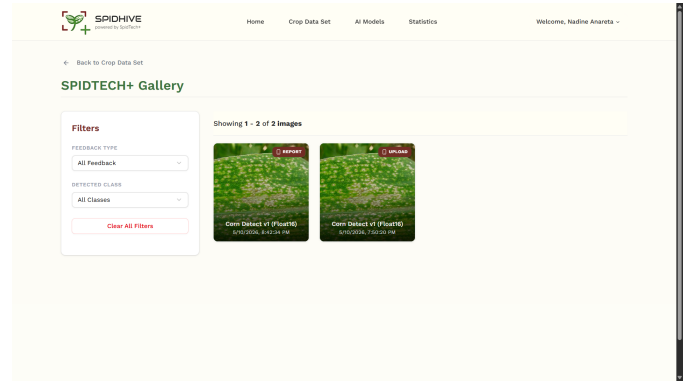


Fig. 14. SPIDTECH+ Gallery

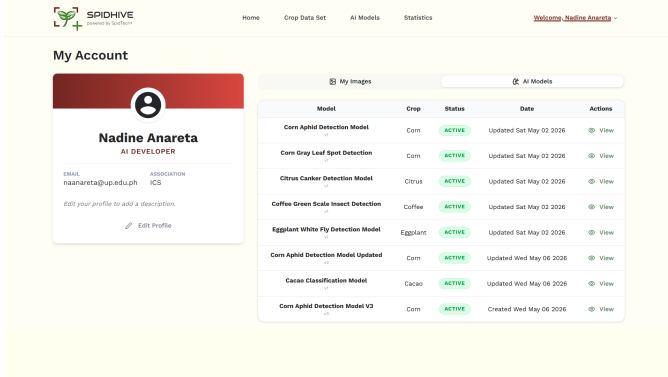


Fig. 13. My Account Page

aries, the interface dynamically renders tools based on user authorization: "Contributors" are restricted to the My Images and My Annotations tabs for raw data handling, whereas "AI Developers" access My Images and a specialized AI Models tab. To protect expert-verified data, the labeling interface is completely hidden from developers; instead, their profile focuses on comprehensive model management, featuring a secure recovery function for restoring archived artifacts.

5) *SPIDTECH+ Gallery*: Figure 14 illustrates the SPIDTECH+ gallery interface, which mirrors images uploaded and reported via the mobile application directly into SPIDHIVE to facilitate continuous dataset enhancement and AI model retraining. These images are fetched from a Supabase storage bucket and displayed alongside comprehensive metadata, including the creation date, feedback type (upload or report), the specific AI model used, user evaluations, and exact image attributes such as size, file path, target crop, and photo validity. To maintain strict data integrity before these images are used for future model training, only authorized expert contributors are granted the permission to modify the crop classifications, labels, and bounding box annotations associated with these mobile submissions.

Across the platform, various UI/UX elements underwent systematic refinement to establish a modern aesthetic and ensure functional consistency. However, the core architectural layouts and navigational hierarchies were deliberately pre-

served. This strategic design decision was made to minimize cognitive load, ensuring that existing users can seamlessly adopt the new AI model management features without facing a disruptive learning curve.

E. Cross-Platform Integration and API Architecture

The management and publication of new AI models necessitate a robust API to handle complex data operations, including fetching metadata, downloading artifacts, and uploading new models. Table 4 details the specific RESTful API endpoints engineered to manage the AI Model Zoo's lifecycle.

TABLE IV
API ENDPOINTS IN AI MODEL ZOO

Field	Description
GET /	Fetch all AI Models
GET /:id	Fetch the details of a specific AI Model
GET /:id/versions	Traverses the branching lineage of the AI Model, fetching both the descendants and ancestors
GET /download/:id	Downloads the .tflite file of an AI Model
GET /download-source/:id	Downloads ZIP file of the entire training folder
DELETE /:id	Performs soft delete on the model
PUT /:id	Edits the details of a model including its name, description, architecture, search tags, parent, and linked dataset
POST /analyze-upload	Extracts and scans the uploaded model folder or single file
POST /publish	Finalizes the uploaded folder/file and uploads it onto the storage
POST /:id/restore	Restores the deleted model

Rather than functioning solely as a traditional web backend, the system's API is architected to serve as a bidirectional distribution hub linking the SPIDHIVE web repository with the SPIDTECH+ mobile application. This cross-platform integration ensures that AI models developed and evaluated on the web are immediately accessible for mobile deployment, while conversely, the images uploaded and reported by users on the mobile app are instantly mirrored back to the web platform for expert annotation and dataset enhancement.

To achieve this seamless synchronization, Supabase is integrated as the dedicated mobile backend service. For model

deployment, this architecture allows the SPIDTECH+ application to securely query the Supabase registry for the latest available models without requiring manual software updates to the application itself. The distribution endpoints are specifically optimized to package and deliver mobile-ready training artifacts, notably the compiled `.tflite` model files. Simultaneously, Supabase storage buckets are utilized to capture the gallery, specifically the raw images and metadata submitted from the field, allowing the SPIDHIVE backend to efficiently fetch and display them for review. By offloading both the delivery of heavy model artifacts and the ingestion of mobile image uploads to Supabase, the architecture is engineered to minimize payload delivery times, prevent bottlenecking on the primary web server, and facilitate highly efficient over-the-air synchronization between the repository and end-user devices.

F. Usability Review

To evaluate the platform’s functional efficiency, a purposive sample of five ($n = 5$) expert AI Developers was utilized, aligning with Nielsen’s model asserting that five representative users can uncover approximately 85% of core usability friction points [24]. Participants executed standard repository workflows—such as uploading training artifacts and interpreting performance analytics—using the Concurrent Think-Aloud (CTA) protocol to continuously verbalize their real-time cognitive processes and navigational expectations [25]. Following these practical tasks, participants completed a System Usability Scale (SUS) adapted from Neil Turner’s framework [26] to systematically quantify subjective user satisfaction and interface clarity. Together, the qualitative CTA insights and structured SUS feedback provided a comprehensive assessment of the AI Model Zoo’s operational readiness.

IV. RESULTS

A. System Implementation and Bidirectional Synchronization

The development of the AI Model Zoo successfully translated complex backend logic into a highly functional web interface. During system evaluation, the automated extraction routines reliably parsed uploaded `.zip` training artifacts to auto-populate neural network metadata, significantly reducing developer cognitive load. Furthermore, the system successfully executed automated dataset processing, accurately partitioning images and generating the required `.yaml` configurations to prevent class-confusion during iterative retraining.

Crucially, Figure 15 illustrates the reliable bidirectional, cross-platform synchronization by integrating Supabase as a dedicated intermediary backend. For downstream deployment, the web platform successfully packaged and delivered compiled `.tflite` AI models to the SPIDTECH+ application for offline edge inference. Conversely, for upstream data collection, telemetry images captured via the mobile app were accurately mirrored onto the SPIDHIVE web dashboard, allowing authorized experts to review and edit field data without bottlenecking the primary AWS server. Ultimately, this architecture successfully establishes a continuous, real-world pipeline between remote field data collection and centralized AI model refinement.

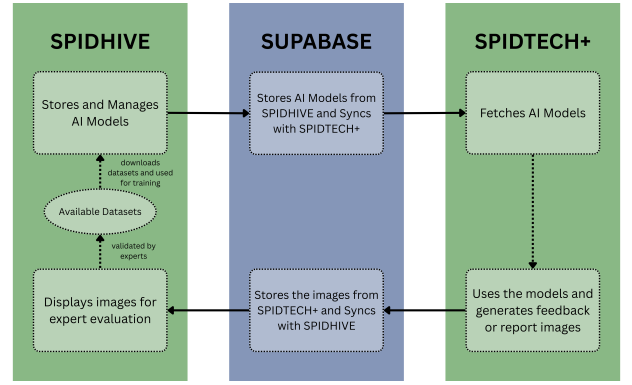


Fig. 15. Bidirectional Synchronization

B. System Usability Scale (SUS)

Following the practical testing sessions, participants completed a standard 10-item SUS questionnaire to measure perceived usability, rating each question on a scale from 1 (Strongly Disagree) to 5 (Strongly Agree).

- 1) I think that I would like to use this system frequently.
- 2) I found the system unnecessarily complex.
- 3) I thought the system was easy to use.
- 4) I think that I would need the support of a technical person to be able to use this system.
- 5) I found the various functions in this system were well integrated.
- 6) I thought there was too much inconsistency in this system.
- 7) I would imagine that most people would learn to use this system very quickly.
- 8) I found the system very cumbersome to use.
- 9) I felt very confident using the system.
- 10) I needed to learn a lot of things before I could get going with this system.

The responses were calculated using the standard SUS scoring framework, yielding a final average system score of 88.0 out of 100. Table 5 illustrates the individual scores, which were consistently high (ranging from 85.0 to 92.5), demonstrating uniform agreement among the testers.

TABLE V
SUS EVALUATION ON ($n = 5$)

#	Raw Score										Final
1	5	2	4	2	4	1	5	1	5	2	87.5
2	5	1	4	2	5	1	5	1	5	2	92.5
3	4	2	5	1	5	1	5	1	4	2	90.0
4	5	1	5	1	5	1	4	1	3	4	85.0
5	5	1	5	4	5	1	5	1	5	4	85.0
Final Average											88.0

According to established SUS benchmarking standards, a score above 68 is considered above average, and a score above 80.3 represents the top 10% of user interfaces [26]. The system’s score of 88.0 places the AI Model Zoo firmly in the “Excellent” category, indicating a very high level of

user satisfaction and a minimal learning curve. Notably, testers scored the platform exceptionally high on integration and learnability, validating the design decision to align the new module's architecture strictly with the existing SPIDHIVE user interface.

C. System Strengths and Refinements

While the SUS score provides a standardized usability benchmark, the open-ended feedback and Concurrent Think-Aloud sessions yielded critical insights into the developers' real-time interactions with the system. The qualitative feedback was thematically analyzed and categorized into core operational strengths and minor areas for iterative refinement.

1) Core Operational Workflows:

- **Workflow Automation:** Participants consistently praised the automated extraction routines during the model upload process. Testers noted that the auto-parsing of .zip artifacts and the integrated documentation generation significantly reduced manual data entry and cognitive load.
- **Information Architecture and User Interface:** The structural layout and visual design received highly positive feedback. Developers found the interface highly intuitive, noting that the modular, tabbed design effectively segregates complex contextual data, making it easy to comprehend even for non-technical users. Furthermore, testers appreciated the responsive UI micro-interactions, such as hover states and smooth animations, which contributed to a modern, professional user experience.
- **System Security and Data Privacy:** A critical validation of the backend architecture came from technical observations regarding API data fetching. Expert testers specifically noted and appreciated the optimized payload design, confirming that the backend successfully sanitizes fetched data to prevent the exposure of non-public variables and sensitive system information.
- **Retrieval and Navigation** Testers confirmed that the repository's search and filtering mechanisms were highly functional, allowing them to easily query and isolate specific AI models from the database without experiencing lag or system errors.

2) **Identified Friction Points and Minor System Refinements:** Because the core functionalities were already highly stable, the actionable recommendations provided by the developers focused strictly on minor, iterative optimizations to further enhance data density and edge-case navigation:

- **UI Density and Context:** To optimize visual scanning, testers recommended slightly reorganizing the AI model preview cards to reduce visual cramping. Additionally, while the upload parsing was praised, developers suggested adding brief explanatory helper text below specific inputs (such as dataset linking) to guide first-time users.
- **Search Filtering Logic:** While the filtering system was deemed highly effective, testers recommended a minor logic adjustment, shifting the underlying query from a strict boolean AND parameter to a broader scope, to

prevent overly limited search results during complex queries.

- **Granular Inspection:** For advanced model evaluation, testers requested the addition of an image zoom function during the upload phase, and recommended that smaller analytical charts be made interactive (clickable or expandable) to ensure strict legibility.

V. CONCLUSION AND FUTURE WORK

A. Conclusion

This study successfully developed the AI Model Zoo module for SPIDHIVE, establishing a centralized lifecycle management platform and a lightweight deployment pipeline for agricultural AI artifacts. By centralizing TensorFlow Lite models, evaluation metadata, and deployment resources, the system resolves conventional dissemination limitations and enables interoperable, cloud-based synchronization with the SPIDTECH+ mobile client. This architecture significantly improves deployment consistency and equips the mobile client with offline, edge-based inference capabilities suitable for low-connectivity agricultural environments. Ultimately, the developed platform bridges the gap between AI model development and practical field deployment, demonstrating the feasibility of integrating robust repository management with mobile synchronization workflows.

B. Limitations

Furthermore, due to the module's significant architectural complexity, it is currently retained in a secure, pre-production staging environment rather than the live public server. Finally, a practical constraint of utilizing managed cloud services, such as Supabase for mobile synchronization and AWS for web hosting, is the recurring financial cost associated with scaling data storage and bandwidth to accommodate nationwide agricultural utilization.

C. Recommendations

The primary recommendation for future work is the phased integration and official deployment of this module into the live SPIDHIVE production server. This transition will require careful database schema migration, comprehensive load testing, and a rigorous cost-benefit analysis of the cloud infrastructure to ensure that the managed services, such as Supabase, can reliably and affordably handle public traffic without disrupting existing operations.

REFERENCES

- [1] P. S. Authority, "Value of production in philippine agriculture and fisheries increased by 5.7 percent in the second quarter of 2025 (at constant 2018 prices)," 8 2025. [Online]. Available: <https://psa.gov.ph/statistics/vop-main/national>
- [2] J. Dogello and U. Cagasan, "A review on the status of crop production innovations of the philippines," pp. 130–136, 11 2021.
- [3] E. O. Brown, "The current state, challenges and plans for philippine agriculture," Nov. 2018. [Online]. Available: <https://ap.iftc.org.tw/article/500>
- [4] M. A. O. Balendres, *Plant diseases caused by fungi in the Philippines*, Jan. 2023, p. 163–188. [Online]. Available: <https://doi.org/10.1016/b978-0-323-99489-7.00014-7>

- [5] B. L. Sullivan, J. L. Aycrigg, J. H. Barry, R. E. Bonney, N. Bruns, C. B. Cooper, T. Damoulas, A. A. Dhondt, T. Dietterich, A. Farnsworth, D. Fink, J. W. Fitzpatrick, T. Fredericks, J. Gerbracht, C. Gomes, W. M. Hochachka, M. J. Iliff, C. Lagoze, F. A. La Sorte, M. Merrifield, W. Morris, T. B. Phillips, M. Reynolds, A. D. Rodewald, K. V. Rosenberg, N. M. Trautmann, A. Wiggins, D. W. Winkler, W.-K. Wong, C. L. Wood, J. Yu, and S. Kelling, "The ebird enterprise: An integrated approach to development and application of citizen science," *Biological Conservation*, vol. 169, p. 31–40, Nov. 2013. [Online]. Available: <https://doi.org/10.1016/j.biocon.2013.11.003>
- [6] A. Guiam, R. Gutierrez, C. V. Gapasin, R. Matalog, and M. Ebuenga, "Smarter pest identification technology (spidtech): a mobile application for digital identification and remote monitoring of insect pests and diseases of major crops in the philippines," *The Philippine Journal of Science*, vol. 150, no. 6B, Nov. 2021. [Online]. Available: <https://doi.org/10.56899/150.6b.17>
- [7] R. Belleza and V. R. Madrid, "Spidtech+: A citizen science approach to building a crop pest and disease index using android," 2025.
- [8] C. C. Commission, "Agriculture," 2017. [Online]. Available: <https://niccdies.climate.gov.ph/mitigation/agriculture>
- [9] I. R. R. Institute, "Philippines introduces innovative satellite-based crop insurance and agro-advisory services to help build farmers' resilience," 2025. [Online]. Available: <https://www.irri.org/news-and-events/news/philippines-introduces-innovative-satellite-based-crop-insurance-and-agro>
- [10] O. Kolesnikova, M. G. Yigezu, A. L. Tonja, M. M. Woldeyohannis, G. Sidorov, and A. Gelbukh, *Ginger disease detection using a computer vision pre-trained model*, 1 2023.
- [11] J. L. Miranda, B. D. Gerardo, and B. T. Tanguilig, III, "Pest detection and extraction using image processing techniques," *International Journal of Computer and Communication Engineering*, vol. 3, no. 3, pp. 189–192, 1 2014. [Online]. Available: <https://doi.org/10.7763/ijcce.2014.v3.317>
- [12] J. M. Bengochea-Guevara, H. Montes, D. Andújar, A. V. Barra, M. N. Conejero, and A. Ribeiro, "Drive-by-wire automation and field validation of a commercial electric vehicle for autonomous crop monitoring," *Smart Agricultural Technology*, vol. 12, p. 101530, Oct. 2025. [Online]. Available: <https://doi.org/10.1016/j.atech.2025.101530>
- [13] J. V. T. Medalla, "Application of wavelet technique in image fusion and its introduction as an early detection tool for spreading of plant pests in philippines' agricultural sector: initial stage," *2018 IEEE 10th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment and Management (HNICEM)*, vol. 10, p. 1–8, Nov. 2018. [Online]. Available: <https://doi.org/10.1109/hnicem.2018.8666424>
- [14] IBM, "Ai model," 6 2025. [Online]. Available: <https://www.ibm.com/think/topics/ai-model>
- [15] B. Zhang, B. Shan, A. Li, C. Luo, Y. Ye, and Z. Li, "ZooKT: Task-adaptive knowledge transfer of Model Zoo for few-shot learning," *Pattern Recognition*, vol. 158, p. 110960, 9 2024. [Online]. Available: <https://doi.org/10.1016/j.patcog.2024.110960>
- [16] E. M. Raouhi, M. Lachgar, H. Himech, and A. Kartit, "Optimization techniques in deep convolutional neuronal networks applied to olive diseases classification," *Artificial Intelligence in Agriculture*, vol. 6, pp. 77–89, 1 2022. [Online]. Available: <https://doi.org/10.1016/j.aiia.2022.06.001>
- [17] Y. Lu, X. Yu, Z. Hu, and X. Wang, "Convolutional neural network combined with reinforcement learning-based dual-mode grey wolf optimizer to identify crop diseases and pests," *Swarm and Evolutionary Computation*, vol. 94, p. 101874, 2 2025. [Online]. Available: <https://doi.org/10.1016/j.swevo.2025.101874>
- [18] A. Bijlwan, R. Ranjan, S. Pokhariyal, A. Govind, M. Singh, K. P. Singh, R. K. Singh, R. K. S. Rajput, and R. K. Srivastava, "Enhancing rice disease and insect-pest detection through augmented deep learning with transfer learning techniques," *Smart Agricultural Technology*, vol. 11, p. 100954, 4 2025. [Online]. Available: <https://doi.org/10.1016/j.atech.2025.100954>
- [19] E. L. Mique and T. D. Palaoag, "Rice Pest and Disease Detection Using Convolutional Neural Network," *ResearchGate*, pp. 147–151, 4 2018. [Online]. Available: <https://doi.org/10.1145/3209914.3209945>
- [20] J. G. Tallada, "Precision agriculture for rice production in the Philippines," 7 2020. [Online]. Available: <https://ap.fttc.org.tw/article/1416>
- [21] N. Magnaye, M. D. Gerance, K. Sta, and J. Fabito, "SaBaTech: A Banana Fruit Pest and Disease Detection Web Application," *American Journal of Data Science and Artificial Intelligence*, vol. 1, no. 1, 6 2025. [Online]. Available: <https://doi.org/10.13140/rg.2.2.25718.31045>
- [22] E. Masicat and V. R. Madrid, "CROPDEX Web Application: Real-Time monitoring and reporting tool for crop pests and diseases," 2025.
- [23] C. India, "What is an ai model library? a beginner's guide to ready-to-use ai models," Nov. 2025. [Online]. Available: <https://community.nasscom.in/communities/ai/what-ai-model-library-beginners-guide-ready-use-ai-models>
- [24] J. Nielsen, "Why you only need to test with 5 users," Feb. 2024. [Online]. Available: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>
- [25] K. Moran, "Usability (user) testing 101," Mar. 2026. [Online]. Available: <https://www.nngroup.com/articles/usability-testing-101/>
- [26] P. Sauro, Jeff, "Measuring usability with the system usability scale (sus) – measuringu." [Online]. Available: <https://measuringu.com/sus/>